

CS 2053M 计算机系统 III

2023–2024 春夏学期期末回忆卷

整理自 CC98 论坛 (linsaiya)，题目不全，仅供参考

题型：选择题 $15 \times 2' = 30'$ ，大题 $6' + 14' + 10' + 25' + 15' = 70'$

一、选择题（共 15 道，每题 2'）

1. (TLB 相关) 下列关于 TLB 的说法，错误的是？

- A. TLB 是页表的专用 cache，缓存最近用过的 PTE
- B. TLB 命中则地址翻译跳过页表 walk，延迟大幅降低
- C. TLB 条目包含 VPN、PPN、保护位等
- D. TLB miss 时不触发任何异常，硬件直接处理完毕

解：D。TLB miss 在某些架构上由硬件页表 walker 自动处理（如 x86），但在 RISC-V 等架构上 TLB miss 会触发 page fault 异常由 OS 处理。“不触发任何异常”太绝对。[lec-11]

2. (Page size 相关) 以下哪项会因页尺寸**变小**而受益？

- A. 时间局部性和空间局部性
- B. TLB 覆盖范围
- C. 虚拟内存分页的内部碎片
- D. 预调页 (Prepaging)

解：C。小页 $\rightarrow$ 内部碎片小（平均每进程浪费 $\frac{1}{2}$ 页）。A/B/D 均倾向大页（更大 TLB reach、更少 page fault、预调页更有效）。[lec-09]

3. (DMA 相关) 下列关于 DMA 的说法，正确的是？

- A. DMA 需要 CPU 逐字节参与数据搬运
- B. DMA 使外设可直接读写主存，释放 CPU 去干别的
- C. DMA 只能用于磁盘 I/O，不能用于网卡
- D. DMA 传输期间 CPU 必须暂停所有操作

解：B。DMA 控制器在外设和主存间直接搬数据，CPU 只在启动和结束时介入。可用于磁盘、网卡、显卡等任何需要批量传输的外设。[lec-13]

4. (UMA/NUMA 相关) 以下关于 UMA 和 NUMA 的说法, **错误**的是?

- A. UMA 中所有 CPU 访问所有内存的延迟相同
- B. NUMA 中 CPU 访问本地内存比远程内存快
- C. UMA 扩展性优于 NUMA, 适合大规模多处理器
- D. NUMA 需要 OS 感知内存拓扑以优化分配

解: C。UMA 的总线/交叉开关在 CPU 数量增多时成为瓶颈, NUMA 通过分布式内存实现更好的可扩展性。[lec-12,13]

5. (Cache 映射方式) 对于相同容量的 Cache, 以下哪种映射方式的**冲突缺失最少**?

- A. 直接映射 (Direct Mapped)
- B. 2-路组相联
- C. 4-路组相联
- D. 全相联 (Fully Associative)

解: D。全相联无冲突缺失 (任一 block 可放任意位置)。关联度越高冲突越少。[lec-04,05]

6. (写策略) 采用写回 (write-back) + 写分配 (write-allocate) 的 Cache 在执行 write miss 时的行为是?

- A. 直接写主存, 不调入 Cache
- B. 先将 block 调入 Cache, 再在 Cache 中写
- C. 只写 Cache, 不写主存, 也不调入 block
- D. 既写 Cache 又同时写主存

解: B。写分配: write miss 时先将 block 读入 Cache, 再修改。写回: 只更新 Cache (标 dirty), 替换时才写回主存。[lec-05]

7. (五级流水线)在经典五级流水线中,以下哪条指令序列会同时产生 RAW hazard 和 control hazard (无 forwarding、无分支预测)?

- A. ADD x1,x2,x3; SUB x4,x1,x5
- B. LW x1,0(x2); ADD x3,x1,x4; BEQ x3,x0,L1
- C. ADD x1,x2,x3; ADD x4,x5,x6
- D. BEQ x1,x2,L1; NOP; NOP

解: B。LW → ADD 之间有 RAW (x1); ADD → BEQ 之间有 RAW (x3) 且 BEQ 导致 control hazard。[lec-02]

8. (Cache 性能) 某程序 CPI 为 2.0, 其中 30% 为 load/store 指令, cache miss rate 为 5%, miss penalty 为 50 周期。若将 miss rate 降至 2%, 新的 CPI 约为?

解: 原 $\text{stall} = 0.3 \times 0.05 \times 50 = 0.75$, $\text{CPI}_{\text{old}} = 2.0 + 0.75 = 2.75$ 。

新 $\text{stall} = 0.3 \times 0.02 \times 50 = 0.30$, $\text{CPI}_{\text{new}} = 2.0 + 0.30 = \mathbf{2.30}$ 。[lec-07]

9. (SIMD/向量计算, 原卷第 9 题) 计算以下代码的循环体所需周期数。a, b, c 均为 1 byte 元素的数组。

```
for (i = 0; i < 524; i++) {  
    c[i] = a[i] + b[i];  
}
```

已知 ALU 的 8-bit 加法需 1 周期, 64-bit 加法需 4 周期。采用 64-bit 加法一次处理 8 个 byte。

- A. 264
- B. 268
- C. 524
- D. 48

解: A。[524/8] = 66 次 64-bit 加法, $66 \times 4 = \mathbf{264}$ 周期。[lec-18]

10. (页表相关) 某系统 32-bit 虚拟地址, 4KB 页面, 采用两级页表, 每级 1024 项 (每项 4B)。页表本身占多少物理内存? 两级各多大?

解: 每级 $1024 \times 4 = 4\text{KB}$ (恰好一页)。两级合计 8KB, 但未用满的页表分支可不分配。[lec-09]

11. (Page Fault 相关, 原卷第 13 题) 假设有 3 个页框, 初始为空。引用串: 2, 1, 3, 4, 2, 1, 5, 6, 2, 1, 7, 3, 4, 2, 3。用 FIFO 替换算法, 共发生几次 page fault?

- A. 11
- B. 12
- C. 13
- D. 14

解: D。FIFO 模拟 (3 页框):

访问	页框 (FIFO 队列)	PF
2	[2,_,_]	1
1	[2,1,_]	2
3	[2,1,3]	3
4	[4,1,3]	4 (evict 2)
2	[4,2,3]	5 (evict 1)
1	[4,2,1]	6 (evict 3)
5	[5,2,1]	7 (evict 4)
6	[5,6,1]	8 (evict 2)
2	[5,6,2]	9 (evict 1)
1	[1,6,2]	10 (evict 5)
7	[1,7,2]	11 (evict 6)
3	[1,7,3]	12 (evict 2)
4	[4,7,3]	13 (evict 1)
2	[4,2,3]	14 (evict 7)
3	[4,2,3]	hit

共 **14 次** page fault。[lec-10]

12. (缓存 vs TLB) TLB 是?

- A. 一个页表
- B. 一个缓存主存数据的 buffer (cache)
- C. 一个缓存虚拟地址到物理地址映射关系 (PTE) 的 buffer (cache)
- D. 一种页替换算法

解: C。TLB (Translation Lookaside Buffer) 是页表的专用 cache, 缓存最近使用过的 PTE (VPN → PPN 映射), 加速地址翻译。它不存数据内容 (B 错), 不是页表本身 (A 错), 也不是替换算法 (D 错)。[lec-11]

13. (综合判断) 以下关于虚拟内存的说法, **正确**的是?

- A. 虚拟内存的大小由物理内存容量决定
- B. 每个进程有自己独立的虚拟地址空间
- C. 虚拟内存只能用于磁盘交换, 不能用于其他目的
- D. 虚拟地址和物理地址总是一一对应

解: B。虚拟地址空间大小由 CPU 位宽决定 (如 48-bit), 与物理内存无关。虚拟内存的核心是地址空间隔离 + 按需调页, 不只限于 swap。多个虚拟页可映射到同一物理页 (共享)。[lec-10]

14. (综合判断) 以下关于 Tomasulo 算法的说法, **错误**的是?

- A. Register Rename 通过 Reservation Station 和 ROB 实现

- B. Tomasulo 可消除 WAR 和 WAW hazard
- C. Scoreboard 和 Tomasulo 都支持 Speculation
- D. ROB 保证精确异常和按序提交

解：C。Scoreboard 不支持 speculation——它不做分支预测，也没有 ROB。只有 Tomasulo + ROB 才支持 Hardware-Based Speculation。[lec-03]

二、大题

1. Cache 性能与 Speedup 计算 (6')

当全部 Cache hit 时, CPI 为 1.0。指令中有 50% 的 ld/sd 指令。Cache 在 ld/sd 上的 miss rate 为 3%, miss penalty 为 20 周期。当指令全部 Cache hit 时, 求 Speedup。

解:

$$\text{Memory stall cycles} = \text{IC} \times 0.5 \times 0.03 \times 20 = 0.3 \cdot \text{IC}$$

$$\text{CPI}_{\text{actual}} = 1.0 + 0.3 = 1.3, \quad \text{CPI}_{\text{ideal}} = 1.0$$

$$\text{Speedup} = \frac{\text{CPI}_{\text{actual}}}{\text{CPI}_{\text{ideal}}} = \frac{1.3}{1.0} = 1.3$$

[lec-07]

2. Scoreboard / Hardware-Based Speculation (14')

运行以下指令。ld 需 1 周期, fadd/fsb 需 2 周期, fdiv 需 20 周期。

```
fld    f6, 34(R2)
fld    f2, 45(R3)
fadd.d f6, f8, f2
fdiv.d f10, f0, f4
fsub.d f0, f2, f8
```

(原卷后续还有 3 条类似指令, 此处按回忆还原)

- (1) 找出指令中的 WAW 和 WAR。
- (2) 指令中的 WAW 和 WAR 在 Scoreboard 中会导致 stall 吗? 在 Hardware-Based Speculation (Tomasulo + ROB) 中呢?
- (3) 在 Hardware-Based Speculation 下 (假设每种 FU 有独立 RS), 写出每条指令各阶段对应的 Cycle。

解:

- (1)
 - **WAW**: fld f6 和 fadd.d f6 都写 f6。
 - **WAR**: fdiv.d f10, f0, f4 读 f0, fsub.d f0, f2, f8 写 f0。
- (2)
 - **Scoreboard**: 会 stall。Scoreboard 不做寄存器重命名, WAW 和 WAR 都需等待前序指令完成写入后才能发射后续指令。
 - **Tomasulo + ROB**: 不会 stall。寄存器重命名将 f6 和 f0 映射到不同物理寄存器/ROB 条目, WAW 和 WAR 被消除。

(3) 在 HW Speculation 下 (各 FU 独立 RS)，指令在 ROB 中按序提交。

inst	operands	issue	execute	write result	commit
fld f6,34(R2)	—	1	1	2	2
fld f2,45(R3)	—	2	2	3	3
fadd.d f6,f8,f2	f8,f2	4	4–5	6	6
fdiv.d f10,f0,f4	f0,f4	5	5–24	25	25
fsub.d f0,f2,f8	f2,f8	7	7–8	9	26

说明：

- fadd.d 的 f2 在 c3 ready, f8 假设就绪, issue@c4。
- fdiv.d 与 fadd.d 使用不同 FU, 可 c5 并行发射 (无 WAW, 因重命名)。
- fsub.d 等 FP adder RS 空出 (fadd.d WB@c6 释放) 后在 c7 发射。
- fsub.d WB@c9, 但 ROB 中排在 fdiv.d (ROB#4) 之后; fdiv.d 直到 c25 才 WB/Commit, 因此 fsub.d 的 Commit 被 **阻塞到 c26** (ROB 按序提交)。

这是 Tomasulo + ROB 的经典考点: WB 可乱序, 但 **Commit 必须按序**, 长延迟指令会阻塞后续已完成的指令提交。[lec-03]

3. Cube / 互连网络相关计算 (10')

(原卷回忆缺失具体题目描述, 仅记为 “Cube 相关计算……完全没预习这一部分”。以下根据 SYS3 课内知识点重构。)

解: 知识点: 超立方体 (Hypercube) 互连网络。

n 维超立方体: $N = 2^n$ 个节点, 每个节点度数为 n , 直径 = n , 总边数 = $n \cdot 2^{n-1}$, 对分宽度 = 2^{n-1} 。

路由: 从源到目的, 逐位翻转不同的二进制位 (XOR 决定路径)。[lec-18,19] **课件无独立章节, 疑为补充内容**

4. Virtual Memory Paging (25')

(1) 页大小 4KB, 给出 page table (0–15 共 16 项), page table 高位为 1 代表该 page 有效。给定三个 VA, 求对应的 PA 或标注 invalid address。

VA: 0x0000b9d5 (decimal: 47573)

VA: 0x00006e31 (decimal: 28209)

VA: 0x00002685 (decimal: 9861)

(原卷此处附有 16 项页表的具体值, 回忆缺失。以下为解题方法, 数据仅供示意。)

解: 4KB 页 $\rightarrow$ offset = 12 bit, VPN = 4 bit (16 项)。

以 0x0000b9d5 为例: VPN = 0xb (高 4 位), offset = 0x9d5。查页表第 11 项, 若 valid, PA = (PFN $\ll$ 12) | offset; 若 invalid $\rightarrow$ page fault。

- (2) (a) Physical memory 有 128 个 page, 每 page 32 byte。求 VA、PA 各自的总位数、page offset 位数、VPN/PFN 位数。
- (b) 128 个 page 的内容如下 (原卷此处省略 3 页数据)。其中 page 108 为 VA 输入的第一级地址。给出两个 16-bit VA, 求对应的 PA 的内容值, 或标注 invalid address。

VA: 0x611c

VA: 0x3d95

参考例子:

VA: 0x17c3 -> PTE: 0x05 -> Value: d4

(Valid 1, PFN: 0x54, decimal: 84)

PDE: 0x1e -> Value: 0x9c

(Valid 1, PFN: 0x1c, decimal: 28)

PDE: 0x03 -> Value: 0x1c

解:

- (1) 方法: $VPN = VA \gg 12$ (对于 4KB 页), 查页表得 PFN。若 valid 位为 1, $PA = (PFN \ll 12) | (VA \& 0xFFF)$; 若为 0, invalid。具体数值依赖缺失的页表内容。

- (2) (a) 128 pages $\rightarrow$ PFN 需 $\log_2 128 = 7$ bit。每页 32 byte $\rightarrow$ offset 需 $\log_2 32 = 5$ bit。PA = PFN(7) + offset(5) = **12 bit**。

VA 位数需大于等于 PA, 题目给定 16-bit VA: offset = 5 bit, VPN = 16 - 5 = **11 bit**。

若采用两级页表: VPN[1] 和 VPN[0] 各占若干 bit。

- (b) 两级页表翻译流程 (16-bit VA, 32 B 页 $\rightarrow$ offset 5 bit, VPN 11 bit):

$0x611c = 0110\ 0001\ 0001\ 1100_2$: offset = 0x1c (5 bit), VPN = 0x30c (11 bit), 拆为 PDE index + PTE index。

查 page 108 (PDE 根) $\rightarrow$ 得二级页表 PFN $\rightarrow$ 查二级 PTE $\rightarrow$ 得数据 PFN $\rightarrow$ PA = PFN $\ll 5$ | offset。

与参考例子同理, 具体值依赖题目所给页表数据。[lec-09,10,11]

5. File System (15')

原卷前 3 页介绍了 mkdir(), create(), link() 对文件系统的影响, 随后用一页给出示例操作序列, 要求补写 5 个操作使得文件系统状态与给定结果一致。

解: 考点总结 (基于 UNIX 文件系统语义):

- mkdir("foo"): 创建目录 foo, 分配 inode + 数据块, 在其中写入 . (自身) 和 .. (父目录) 两项。父目录新增条目 foo $\rightarrow$ inode#。
- create("bar"): 创建普通文件 bar, 分配 inode, 父目录新增条目 bar $\rightarrow$ inode#。文件初始为空 (size = 0, 无数据块)。

- `link("old", "new")` (硬链接): 在目标目录中新增条目 `new` $\rightarrow$ `inode#(old)`, `inode` 的 `link count` +1。不分配新 `inode`, 不复制数据。
- `symlink("old", "new")` (符号链接): 创建新文件 `new`, `inode` 标记为符号链接类型, 数据块存目标路径字符串。

解题思路: 追踪每次操作后 `inode` 表 (新分配/复用) 和目录内容的变化, 确保最终状态与目标一致 (硬链接 vs 符号链接的选择是关键区分点)。[lec-14,15,16]